

CASE STUDY ANALYSIS

Cybersecurity Attack on a Major Organization

Introduction

Cybersecurity is an important part of modern software engineering because organizations store large amounts of personal, financial, and business information in computer systems. A security weakness in one application can sometimes give attackers access to many other systems. This case study examines the 2017 cybersecurity attack on Equifax, a major credit reporting organization. Attackers exploited a known vulnerability in the Apache Struts web framework and gained access to Equifax systems containing sensitive personal information. Government investigations later identified weaknesses in patch management, network segmentation, detection, and data protection.

The case is useful from a software engineering point of view because the problem was not only a technical vulnerability. The organization also had problems in identifying the affected system, applying the required security update, monitoring the network, protecting credentials, and limiting access between systems. Therefore, the case can be studied as a complete software engineering and system management problem.

Understand the Problem Statement

Equifax handled a very large amount of sensitive consumer information. Its systems included web applications, databases, and older systems that supported credit-report related services. In March 2017, Equifax received an alert about a critical vulnerability in Apache Struts, a framework used for Java web applications. The vulnerability could allow an attacker to execute commands on a vulnerable server. A vulnerable Equifax online dispute portal was not successfully patched, and attackers later used the weakness to enter the system.

The attackers did not stop after entering the first server. They were able to move through the environment and query databases containing personally identifiable information. Government investigators reported that the attackers used techniques that helped them hide their activity. An expired digital certificate also affected Equifax's ability to inspect network traffic for malicious activity.

Equifax announced the breach in September 2017. The breach affected approximately 147 million people and exposed information such as names, dates of birth, Social Security numbers, addresses, and other sensitive information. The incident became a major example of how weaknesses in software maintenance and security controls can create serious consequences for users and an organization.

The main problem can be stated simply: Equifax had security controls and processes, but they did not work together effectively to identify the vulnerable system, apply the required patch, limit the attacker's movement, and detect unauthorized activity quickly.

Objectives of the Case Study

- Understand how a known software vulnerability was used in a real cyberattack.
- Identify the software engineering and security weaknesses that contributed to the incident.
- Understand the effect of poor patch management and weak system monitoring.
- Study how network segmentation and access control can reduce the impact of an attack.
- Suggest a better software and security approach for preventing similar incidents.
- Show how security should be included during development, deployment, testing, and maintenance.

Scope of the Case Study

The scope covers vulnerability management, patch management, access control, network segmentation, secure data storage, monitoring, incident response, software testing, and maintenance. It does not attempt to reproduce the attack or provide instructions for attacking a real system. The purpose is to understand the failure and propose safer engineering practices.

Identify Key Stakeholders and Challenges

A cybersecurity incident in a large organization affects more than the security team. Many people and groups are connected to the system, from customers and employees to software developers, managers, regulators, and incident-response teams. Each stakeholder has a different responsibility, and weak coordination between them can increase security risk.

Key stakeholders include:

- Customers and Consumers – Their personal information was stored or processed by the organization. They expect the organization to protect their information.
- Security Team – Responsible for vulnerability monitoring, threat detection, security tools, and incident response.
- System Administrators – Responsible for servers, software updates, certificates, configurations, access permissions, and maintenance.
- Software Development and Maintenance Teams – Responsible for applications and third-party frameworks and dependencies.
- Database and Infrastructure Teams – Responsible for databases, access controls, backups, and network design.
- Senior Management – Responsible for security priorities, resources, policies, and risk decisions.
- Legal and Compliance Teams – Responsible for privacy, reporting, and regulatory requirements.

- Government and Regulatory Bodies – Investigate serious incidents and enforce security and privacy requirements.

Major challenges included:

- Large and complex IT infrastructure made complete visibility difficult.
- A known vulnerability remained on an internet-facing application.
- Patch management and follow-up were not effective enough.
- Weak network segmentation allowed attackers to reach additional systems and data.
- Sensitive credentials were stored in an unsafe way.
- An expired digital certificate weakened network monitoring.
- Large amounts of sensitive data increased the impact of the incident.
- Incident response had to deal with a very large number of affected people.

These challenges show that cybersecurity is both a technical and management problem. A technically strong security tool cannot completely protect an organization if software updates are missed, responsibilities are unclear, or important systems are not monitored.

Analyze the Current Approach

For this case study, the current approach refers to the security and software maintenance practices that existed around the time of the Equifax breach. Equifax had security tools, vulnerability notifications, scanning, network monitoring, and processes for applying patches. However, the controls were not working effectively as a complete security process.

The general process can be understood as follows. A software vulnerability is discovered and a security alert is received. The organization identifies affected systems, applies the required patch, scans the environment, and monitors traffic. Access to sensitive information should be restricted using authentication, authorization, network segmentation, and database controls. If suspicious activity is detected, the incident response team investigates and contains it.

In the Equifax case, several parts of this process failed or were not effective enough. The vulnerability existed in the online dispute portal even after the company had been warned about the issue. Network scanning did not successfully identify the vulnerable portal. The attackers later exploited the vulnerable system and accessed additional information.

Strengths of the Existing Approach

- Equifax had a security team that received vulnerability information.
- The organization used vulnerability scanning and network monitoring tools.
- Security staff were able to identify suspicious traffic later in the incident.
- Logs and forensic analysis helped investigators reconstruct attacker activity.
- After the incident, the organization took steps to address weaknesses and support affected users.

Limitations of the Existing Approach

- Security alerts were not reliably connected to the correct system owners.
- The vulnerable software was not successfully patched within the required time.
- Scanning did not identify the vulnerable online dispute portal.
- Network segmentation was not strong enough to prevent movement toward sensitive databases.
- Some administrative credentials were stored in plain text.
- A certificate used by a monitoring tool had expired.
- The organization had difficulty maintaining complete visibility across a large environment.

Problems and Gaps

The biggest gap was the difference between having security controls and making sure that those controls actually work. A patching policy is useful only when every vulnerable system is identified, assigned to an owner, updated, and checked afterward. Similarly, a monitoring tool is useful only when certificates, configurations, logs, and alerts are maintained correctly.

Another important gap was excessive access between systems. Once attackers entered the vulnerable application, they were able to reach other systems and retrieve sensitive data. Strong segmentation and least-privilege access could have reduced the number of systems reachable from the compromised application.

Propose a Suitable Solution Using Software Engineering Principles

A suitable solution is to develop and maintain a security-focused enterprise system in which vulnerability management, patch management, access control, monitoring, testing, and incident response are connected. The purpose is not only to prevent one known vulnerability but to create a repeatable process that continuously checks the security condition of applications and infrastructure.

The proposed solution can be designed as a centralized Cybersecurity Risk and Vulnerability Management System. It can collect information about applications and servers, identify vulnerabilities, assign tasks to responsible teams, verify patches, monitor security controls, and generate alerts when important actions are overdue.

Main Functional Modules

- Asset Management Module – Maintains applications, servers, databases, services, and their owners.
- Vulnerability Management Module – Records vulnerabilities, severity, affected software, and required actions.
- Patch Management Module – Tracks patches from identification to installation and verification.
- Security Scanning Module – Performs authorized vulnerability and configuration checks.
- Access Control Module – Applies authentication, authorization, least privilege, and role-based access.
- Network Segmentation Module – Defines which systems are allowed to communicate.

- Monitoring and Alert Module – Collects security events and raises alerts for suspicious activity.
- Credential Management Module – Protects passwords and sensitive credentials.
- Incident Response Module – Records incidents, assigns tasks, and tracks containment.
- Reporting Module – Provides dashboards and reports for technical teams, management, and audit.

Functional Requirements

- The system shall maintain an inventory of important software and infrastructure assets.
- The system shall record known vulnerabilities and their severity.
- The system shall assign each vulnerability to a responsible owner.
- The system shall record patch status and the date of verification.
- The system shall generate alerts for critical vulnerabilities that remain unresolved.
- The system shall record security events and suspicious activity.
- The system shall support role-based access to security information.
- The system shall create incident records and maintain an audit trail.
- The system shall generate security and compliance reports.

Non-Functional Requirements

- Security – Sensitive information must be protected using strong authentication, authorization, encryption, and secure credential storage.
- Reliability – Security monitoring and alerting should work consistently.
- Performance – Alerts and dashboards should respond within a reasonable time.
- Scalability – The system should support a growing number of assets and events.
- Maintainability – Modules should be separated so changes are easier.
- Usability – Security staff should understand alerts and tasks easily.
- Auditability – Important actions should be recorded.

System Architecture (Block Diagram)

SECURITY USERS Security Team System Administrators Managers Auditors
WEB INTERFACE Login Dashboard Vulnerabilities Patch Status Reports
CYBERSECURITY MANAGEMENT SYSTEM Asset Management Vulnerability Management Patch Management Access Control Monitoring Incident Response Reporting
SECURITY SERVICES Vulnerability Scanner Log Monitoring Alert Service Notification Service

PROTECTED IT ENVIRONMENT Web Applications Servers Databases Network Devices
SECURITY DATABASE Asset Details Vulnerabilities Patch Records Incidents Audit Logs
ORGANIZATIONAL CONTROLS Security Policies Staff Training Backup Disaster Recovery

The architecture follows a layered approach. The web interface provides access to the system. The cybersecurity management system contains the main business logic. Security services provide scanning, monitoring, and alerts. The protected environment contains the organization's applications and infrastructure. The database stores records needed for tracking and auditing. Organizational controls such as training, policies, backup, and disaster recovery support the technical controls.

Software Engineering Principles Used

- Modularity – Security functions are divided into separate modules.
- Scalability – The architecture can add more assets, users, and monitoring data.
- Maintainability – Each module can be updated separately.
- Reliability – Monitoring, backups, logging, and verification improve reliability.
- Security by Design – Security is included from the beginning.
- Least Privilege – Users and services receive only the access they need.
- Defense in Depth – Multiple security controls work together.
- Auditability – Important security actions are recorded.

Justify the Chosen Methodology/Framework

Agile methodology is suitable for developing the proposed cybersecurity management system. Cybersecurity requirements can change because new vulnerabilities, attack methods, software versions, and business requirements appear regularly. Agile development allows the system to be built in small stages, tested frequently, and improved using feedback.

A possible Agile development plan is:

Sprint 1 – Collect requirements, identify assets, define user roles, and design architecture.

Sprint 2 – Develop asset management and user access control.

Sprint 3 – Develop vulnerability tracking and patch management.

Sprint 4 – Add scanning, monitoring, and alert functions.

Sprint 5 – Develop incident response and reporting.

Sprint 6 – Perform security, performance, user, and final testing.

Agile is useful because each sprint can produce a working part of the security system. Security testing can be performed throughout development instead of waiting until the final stage. Feedback from administrators and security officers can improve dashboards, alerts, and workflows.

Agile should also be combined with secure software development practices. Code review, dependency checking, automated testing, vulnerability scanning, secure configuration, and access-control testing should be included in every sprint. Security is not a one-time activity; after deployment, new vulnerabilities must be monitored, assets reviewed, patches verified, and security controls tested regularly.

Discuss Expected Outcomes and Limitations

The proposed solution is expected to reduce the chance that a known vulnerability remains unnoticed or unpatched for a long time. It also improves visibility because security teams can see which assets exist, which vulnerabilities affect them, who is responsible, and whether corrective action has been completed.

Expected outcomes include:

- Faster identification of vulnerable applications and systems.
- Better tracking of security patches from assignment to verification.
- Reduced chance of missing important security alerts.
- Stronger access control and safer credential handling.
- Reduced attacker movement through network segmentation.
- Earlier detection of suspicious activity.
- Better incident response through clear incident records.
- Improved reporting for management and audit teams.
- Better protection of sensitive information.
- Reduced financial, legal, operational, and reputational risk.

The Equifax incident resulted in a major financial and legal impact. The Federal Trade Commission reported that Equifax agreed to pay at least \$575 million and potentially up to \$700 million as part of a global settlement related to the breach. The incident also affected a very large number of consumers.

Risks and Limitations

- No system can provide complete protection against every future cyberattack.
- A centralized security platform can itself become a target.
- Automated scanning can produce false positives and may require expert review.
- Old systems may be difficult to update or replace.
- Security improvements require money, skilled staff, training, and maintenance.
- Network segmentation can affect normal communication if configured incorrectly.
- Employees may make mistakes, so technical controls must be supported by training.
- Internet-connected systems remain exposed to new vulnerabilities.

Future Enhancements

- Use machine learning to identify unusual patterns in security events.
- Add automatic patch verification and configuration checking.
- Integrate threat intelligence feeds for newly reported vulnerabilities.
- Use stronger zero-trust access controls for sensitive systems.
- Add automated backup and recovery checks.
- Provide alerts for critical security incidents.
- Use dashboards showing risk levels and unresolved critical issues.
- Perform regular penetration testing and independent security audits.
- Integrate security checks into software development pipelines.

Conclusion

The 2017 Equifax data breach is an important cybersecurity case study because it demonstrates how a known software vulnerability can become a large-scale incident when patch management, monitoring, network segmentation, access control, and security processes are not effective enough. The breach affected approximately 147 million people and resulted in major financial, legal, and reputational consequences.

From a software engineering perspective, the main lesson is that security must be treated as a continuous part of the system life cycle. A secure application is not created only by writing secure code. The organization must also maintain dependencies, monitor systems, verify patches, protect credentials, control access, test security, and respond quickly to incidents. The most important lesson is simple: finding a vulnerability is not enough; the organization must make sure that it is actually fixed and that the fix has been verified.